

BitForge Development Policies

Policy version: 1.0 | **Date:** 2026-10-04 | **Applies to:** RTL (Verilog-2005), UVM/SV testbench, firmware tools, datasheet, specification, PRD and schedule. **Keywords:** **shall** = mandatory, **should** = expected unless a recorded waiver exists. Every policy rule has an ID (NET- , DSU- , SPC- , CHG-) so reviews and waivers can cite it.

Contents: 1 Net Naming | 2 Datasheet Updating | 3 Technical Specification | 4 Change and Update Management | 5 Waivers and Compliance

1. Net Naming Policy

1.1 General format

ID	Rule
NET-001	Names shall be lowercase snake_case , ASCII letters, digits and _ only; first character a letter
NET-002	Names shall be descriptive and use agreed abbreviations only (Section 1.6); no single-letter nets except loop indices and i / j / k in testbench code
NET-003	Maximum length 40 characters; hierarchy is not repeated in a name (use fifo_full , not lane0_fifo_fifo_full)
NET-004	Names shall not differ only by case, and shall not be Verilog/SystemVerilog keywords or reuse a name already used in the same module scope
NET-005	Parameters and localparam/define constants shall be UPPER_SNAKE_CASE ; module and file names shall be lowercase with the project prefix bf_ (file name equals module name)
NET-006	Instance names shall be u_<module_suffix> (e.g. u_alu); lane instances shall be u_lane0 , u_lane1

1.2 Suffix and prefix conventions

ID	Rule	Example
NET-010	Input ports end in _i , output ports in _o , tri-state enable in _oe (active high), bidirectional data split into _i , _o , _oe; no inout nets inside the design	pin_data_i , pin_data_o , pin_data_oe
NET-011	Active-low signals carry _n ; when a port suffix is also needed, _n comes first and the port suffix last. The top-level reset is the exception (NET-012)	fifo_empty_n , cs_n_i
NET-012	Reset: rst_n is the only reset name at top level; internal copies are rst_n_sync	rst_n , rst_n_sync

NET-013	Registers (flip-flop outputs) end in <code>_q</code> ; their next-state combinational inputs end in <code>_d</code> ; enable signals <code>_en</code> ; load signals <code>_ld</code>	<code>pc_q</code> , <code>pc_d</code> , <code>shift_en</code>
NET-014	Synchronizer stages end in <code>_s1</code> , <code>_s2</code> ; the final synchronized signal ends in <code>_sync</code>	<code>pin_in_s1</code> , <code>pin_in_sync</code>
NET-015	Clock signals contain <code>clk</code> ; there is one clock domain (<code>clk</code>); any gated/derived clock needs a waiver (CHG-050)	<code>clk</code>
NET-016	Strobes/pulses end in <code>_pulse</code> or <code>_stb</code> ; flags in <code>_flag</code> ; valid/ready handshakes use <code>_valid</code> / <code>_ready</code> ; counters <code>_cnt</code> ; status registers <code>_stat</code> ; configuration <code>_cfg</code>	<code>rx_push_stb</code> , <code>fault_flag</code>
NET-017	Buses use <code>[MSB:LSB]</code> with LSB 0, descending; width comes from a named parameter, not a literal	<code>[DATA_W-1:0]</code>
NET-018	Per-lane signals: <code>l<N>_</code> prefix when they appear at top level or in shared modules (<code>l0_pc_q</code>); inside a lane module the prefix is dropped	<code>l1_fifo_level</code>
NET-019	Register fields in RTL use the register's name and field: <code><reg>_<field></code> and match <code>regmap.yaml</code> exactly	<code>lane_ctrl_run</code>

1.3 Top-level ports (fixed)

ID	Rule
NET-020	The Tiny Tapeout wrapper port names (<code>ui_in</code> , <code>uo_out</code> , <code>uio_in</code> , <code>uio_out</code> , <code>uio_oe</code> , <code>ena</code> , <code>clk</code> , <code>rst_n</code>) shall be kept exactly as the template defines them; internal signals map to them in one module (<code>bf_top</code>) with documented aliases
NET-021	A pin map table (pin, direction, function, internal net) shall exist in the datasheet and be checked against <code>bf_top</code> by CI

1.4 Testbench and verification names

ID	Rule
NET-030	UVM classes: <code>bf_<role>_<kind></code> in lowercase (<code>bf_spi_host_agent</code> , <code>bf_lane_scoreboard</code>); type suffixes <code>_agent</code> , <code>_driver</code> , <code>_monitor</code> , <code>_seq</code> , <code>_item</code> , <code>_cfg</code> , <code>_test</code> , <code>_cov</code>
NET-031	Interfaces end in <code>_if</code> , virtual interface handles in <code>_vif</code> ; test names start with the area (<code>uart_</code> , <code>spi_</code> , <code>i2c_</code> , <code>rand_</code>)
NET-032	Assertion properties/assertions are <code>p_<what></code> / <code>a_<what></code> / <code>c_<what></code> (property, assert, cover); covergroups <code>cg_<what></code>
NET-033	Firmware labels are lowercase <code>snake_case</code> ; macros in the assembler are <code>.UPPER_CASE</code>

1.5 Synthesis and netlist stability

ID	Rule
NET-040	Names that appear in constraints, SDC, waveform scripts or the datasheet are stable names ; renaming one is a change request (CHG-020, class Minor)
NET-041	Keep hierarchy through synthesis for the lane, FIFO, crossbar and host modules to preserve readable netlists in gate-level debug
NET-042	Auto-generated names (tool-created) are never referenced in scripts

1.6 Approved abbreviations

addr address | cfg config | cnt counter | ctrl control | dly delay | en enable | ev event | fifo | flag | imem instruction memory | isr/osr input/output shift register | lvl level | pc program counter | rx/tx | stat status | ts timestamp | xbar crossbar. New abbreviations shall be added here before use (NET-050).

ID	Rule
NET-050	Any new abbreviation requires an entry in this list in the same commit that first uses it
NET-051	A lint script shall check NET-001, 003, 005, 010, 012, 013 and 017 in CI; violations fail the build unless waived

2. Datasheet Updating Policy

ID	Rule
DSU-001	The datasheet is the external contract of the chip. It shall describe only behavior that the RTL, ISS and tests verify
DSU-002	The datasheet has one owner of record per section (see Technical Specification sections, SPC-002) and one document version
DSU-003	Register tables, pin map, ISA table and timing tables shall be generated from <code>regmap.yaml</code> , <code>isa.yaml</code> and the timing database; they shall not be edited by hand
DSU-004	Version numbering MAJOR.MINOR.PATCH: MAJOR = incompatible change (register address, opcode meaning, pin function), MINOR = compatible additive change, PATCH = editorial
DSU-005	Document status per section shall be one of: Draft, Reviewed, Frozen . Only Frozen sections are published in a release

2.1 Triggers (a datasheet update is required when)

ID	Trigger
DSU-010	Any change to <code>regmap.yaml</code> , <code>isa.yaml</code> , pin function, reset value, clock rate or limit, or instruction timing
DSU-011	A verification result changes a documented limit (e.g. measured fmax, max baud)
DSU-012	A bug fix changes externally visible behavior
DSU-013	A new or removed protocol firmware example is released
DSU-014	Post-synthesis or post-route results change area, timing or power figures

2.2 Update procedure

ID	Rule
DSU-020	An update shall be made in the same change request (CHG-010) as the RTL/spec change that caused it; a change that alters datasheet-visible behavior cannot close without the datasheet diff
DSU-021	Every update adds an entry to the revision history (version, date, CR ID, affected sections, summary)
DSU-022	CI shall regenerate tables from the YAML sources and fail if the committed datasheet differs
DSU-023	Numerical values (clock, latency, timing, area) shall cite their source (test name, report name, tool version)
DSU-024	Review checklist before status moves to Reviewed: consistent names (NET policy), units present, reset values match RTL, example code assembles and passes simulation, links valid
DSU-025	Frozen sections change only through a Major or Minor change request (Section 4) with explicit note in the revision history
DSU-026	Errata discovered after freeze are published in a dated errata list and never silently rewritten

2.3 Release rules

ID	Rule
DSU-030	Each release tag contains the datasheet PDF/HTML, the generated tables and a document hash recorded in the release notes
DSU-031	The datasheet version shall be printed on its title page and in the chip ID/version register (HIF-006) when behavior changes
DSU-032	Final submission requires all sections Frozen and zero open datasheet issues

3. Technical Specification Policy

This section defines **how** every technical specification for a block shall be written. It does not contain a specification itself.

3.1 Structure

ID	Rule
SPC-001	A technical specification is section based : one top-level section per functional block (e.g. lane core, instruction memory, FIFOs, crossbar/IO, host interface, timestamp/events, top level)
SPC-002	Each section has exactly one owner and a status (Draft, Reviewed, Frozen, as in DSU-005)
SPC-003	Each section shall contain the sub-sections in 3.2, in the stated order. A sub-section that does not apply shall remain present and say " Not applicable " with a one-line reason
SPC-004	Section and sub-section numbering is hierarchical (4 . 2 . 3) and stable; other documents reference sections by number and name
SPC-005	Every statement of behavior shall be testable. If it cannot be tested, it shall be labeled as informative
SPC-006	Each normative statement carries an ID <BLOCK>-nnn that links to a PRD requirement ID and to one or more tests or assertions

3.2 Required sub-sections (in this order)

No.	Sub-section	Required content
1	Overview	Purpose of the block, where it sits in the chip, major interfaces, assumptions and out-of-scope items
2	Features	Bullet list of supported capabilities and limits, each with a requirement ID; configuration options and build-profile differences (MIN / STD / MAX)
3	Block Diagram	Diagram of the block with named signals and sub-modules (names per NET policy), data and control paths, clock domain boundary, and external ports
4	IP Modules	Table of RTL modules in the block (module name, file, parameters, purpose, owner), plus any macro or third-party IP (e.g. SRAM macro, standard cells), its source and license
5	Unit/ Channel	The number and kind of replicated units (lanes, FIFOs, pins, flags), how each is indexed, per-unit resources and limits, and sharing/arbitration rules
6	Clock	Clock source, frequency, domain, enable/gating rules, synchronizer requirements for asynchronous inputs, and timing constraints that

apply to the block

- | | | |
|---|-------------------------|---|
| 7 | Reset | Reset source and polarity, synchronous/asynchronous behavior, the reset value of every register and state, pin tri-state state, and recovery behavior after reset |
| 8 | Register Address | Address map: offset, name, width, access type (RW, RO, WO, W1C), reset value, field list with bit positions, side effects; shall be generated from <code>regmap.yaml</code> |
| 9 | Operation | Functional description: state machines, instruction/cycle timing, stall and error conditions, startup/shutdown sequence, programming sequence, worked timing examples and waveform descriptions |

3.3 Content rules

- | ID | Rule |
|---------|--|
| SPC-010 | Sub-section 8 (Register Address) must match the RTL register decode bit-for-bit; CI shall compare them |
| SPC-011 | Sub-section 7 (Reset) shall list every register; a register without a stated reset value is a defect |
| SPC-012 | Sub-section 6 (Clock) shall state the measured or target fmax and cite its source (TIM-001, DSU-023) |
| SPC-013 | Sub-section 3 (Block Diagram) names shall match sub-section 4 module names and RTL instance names |
| SPC-014 | Sub-section 9 (Operation) shall give cycle counts for every instruction or event that has timing guarantees; stall behavior shall be explicit |
| SPC-015 | Units: time in ns/cycles, frequency in Hz with prefix, addresses in hexadecimal (0x), sizes in bits/bytes; each value states its unit |
| SPC-016 | Terminology shall follow the project glossary; the first use of an abbreviation in a section is spelled out |
| SPC-017 | A specification section shall be approved (status Reviewed) before RTL implementation of that block begins, and re-reviewed whenever it changes (Section 4) |

3.4 Traceability

- | ID | Rule |
|---------|--|
| SPC-020 | The traceability chain is PRD requirement → specification statement → RTL → test/assertion → datasheet statement ; the verification plan (UVM-045) shall record each link |
| SPC-021 | A requirement without a specification statement, or a specification statement |

without a test, is reported by the monthly consistency review (CHG-040)

4. Change and Update Management

All changes to information (PRD, specification, datasheet, schedule, register map, ISA, RTL, tests, policies) follow one process. Because the project has one developer, the roles below are **hats worn in sequence**; the separation is by time and checklist, not by person.

4.1 Roles (hats)

Role	Responsibility
Requester	Raises the change and states the reason
Analyst	Performs impact analysis across documents and code
Approver	Decides accept/reject/defer against the classes below
Implementer	Makes the change in all affected artifacts
Verifier	Runs checks and confirms closure on the checklist

4.2 Change classes

Class	Definition	Examples	Approval
Editorial	No change in meaning or behavior	Typos, formatting, diagrams redrawn	Self-check, no cooling-off
Minor	Compatible change inside one block or document	Rename a stable name, add a field description, clarify an operation sequence	Approver hat after impact analysis
Major	Changes behavior, register address, opcode, pin function, requirement, schedule milestone, or cross-block interface	New instruction, changed reset value, dropping a requirement, moving a gate date	Approver hat after impact analysis and a 24-hour cooling-off period
Emergency	Needed to fix a blocking defect	Build broken, tape-out blocker	Immediate; a retrospective CR is written within 48 hours

4.3 Change request (CR) procedure

ID	Rule
CHG-001	All non-editorial changes shall have a CR in the issue tracker with ID CR- <code>nnn</code>
CHG-002	A CR contains: title. class. reason. affected artifacts. affected requirement IDs.

impact analysis (CHG-020), decision, owner, target date, status

- CHG-003 Status flow: **Open** → **Analyzed** → **Approved / Rejected / Deferred** → **Implemented** → **Verified** → **Closed**
- CHG-004 Every commit message references a CR ID or states `editorial`; CI rejects commits that touch controlled files without one
- CHG-010 A CR is **Closed** only when all affected artifacts are updated in the same release: spec section, datasheet (DSU-020), `isa.yaml` / `regmap.yaml`, RTL, tests, vPlan, schedule
- CHG-011 Rejected and deferred CRs are kept with the reason so they are not re-raised without new information

4.4 Impact analysis checklist (required for Minor, Major)

- | ID | Rule |
|---------|---|
| CHG-020 | The analyst shall check and record: affected requirement IDs; spec sections; datasheet sections; registers and opcodes; pin map; names (NET-040); tests, assertions and coverage; firmware examples; ISS and assembler; area/timing/power; schedule and Stretch/Deferred status |
| CHG-021 | Effort estimate in hours and the schedule effect (slips, drops a Stretch item, or none) shall be recorded for every Major CR |
| CHG-022 | A Major CR that increases area must include a new synthesis result and shall not be Approved if it breaks the area budget (PHY-001) without a descoping decision |

4.5 Baselines and freeze levels

- | ID | Rule |
|---------|--|
| CHG-030 | Baselines are Git tags: <code>base-isa-v0.1</code> , <code>base-regmap-v0.1</code> , <code>base-feature-freeze</code> , <code>rel-v1.0</code> . A change to a baselined item is controlled per the class table |
| CHG-031 | Freeze levels follow the project schedule: Spec freeze (ISA and regmap, 10-18): only Major CRs allowed. Feature freeze (12-20): only Minor CRs that fix defects, plus Emergency. Release freeze (01-11): only Editorial and Emergency |
| CHG-032 | No new Stretch (S) item shall be started after the freeze level that follows its phase |
| CHG-033 | A change to this policy is itself a Major CR |

4.6 Review cadence and consistency

- | ID | Rule |
|---------|--|
| CHG-040 | Weekly review (Sunday): list open CRs, hours spent vs plan, slipped items; update the schedule (a schedule change is a Major CR if a gate date moves) |
| CHG-041 | Monthly consistency review: RRD vs spec vs datasheet vs tests (CRG-021) results |

- CHG-041 **Monthly consistency review:** PRD vs spec vs datasheet vs tests (SPC-021); results logged as CRs
- CHG-042 **Gate reviews** at 10-18, 11-08, 11-29, 12-20, 01-04, 01-15 against the exit criteria in the schedule
- CHG-043 Metrics tracked weekly: open CRs by class, CRs per week, requirement-to-test traceability %, coverage %, synthesis area %, fmax

4.7 Document control

- | ID | Rule |
|---------|--|
| CHG-050 | Waivers (for any rule in this document) are recorded as CRs with the rule ID, justification, scope, and expiry date |
| CHG-051 | Controlled documents each carry: document ID, version, date, status, and a revision history table |
| CHG-052 | Version numbers: PRD and schedule use MAJOR.MINOR (increment MINOR for Minor CRs, MAJOR for Major CRs); datasheet follows DSU-004 |
| CHG-053 | When a PRD requirement is revised, the old text is retained in the revision record (redline), with the CR ID that caused it |
| CHG-054 | All controlled files live in the repository; the repository's main branch is protected and changes enter by reviewed pull request (self-review checklist for a solo developer) |

5. Compliance and Waivers

- | ID | Rule |
|---------|---|
| POL-001 | Compliance is checked by CI scripts where possible (NET-051, DSU-022, SPC-010, CHG-004) and by the gate reviews otherwise |